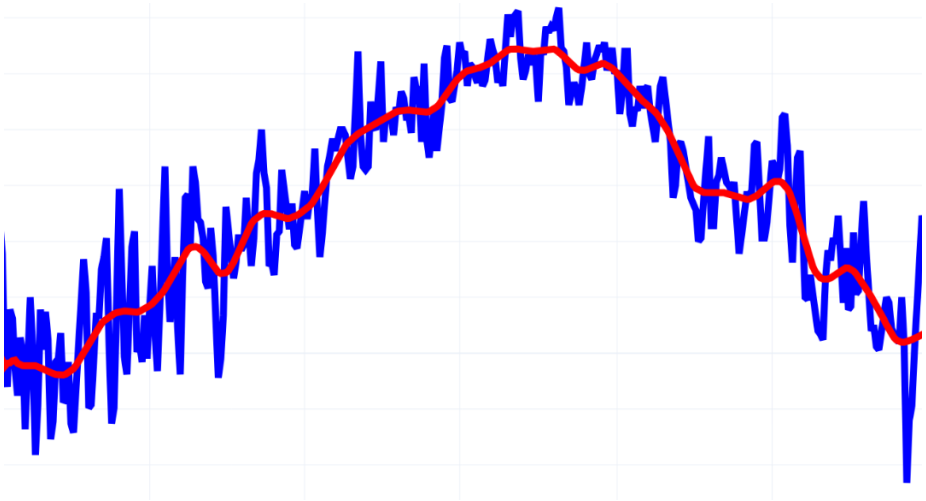
Synthesis of digital filters to remove noise from a music signal

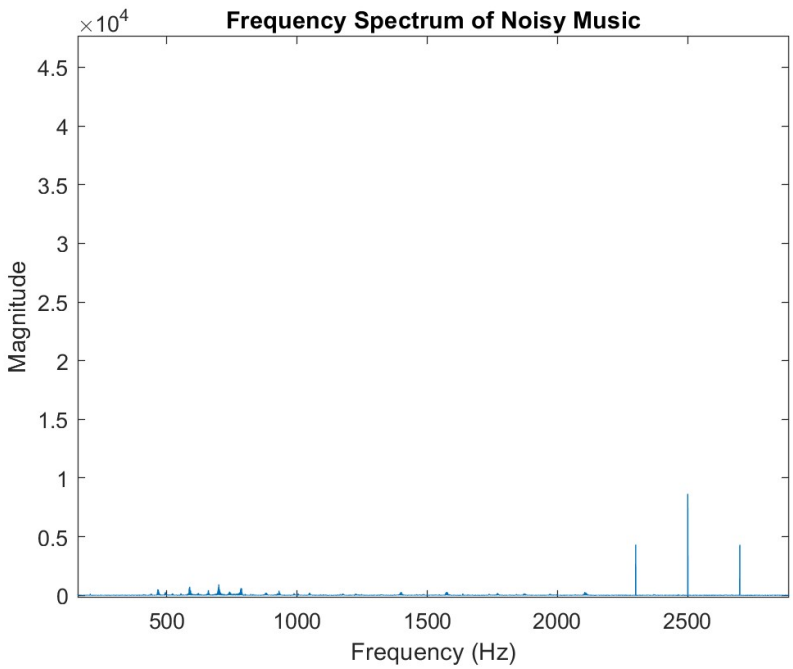
Federico Gabrielli

April 10, 2025



1 Signal Analysis

After analyzing the signal spectrum in MATLAB and verifying the results in Simulink using a Spectrum Analyzer block, we identified the frequency range containing the useful music and determined where the noise frequencies begin.



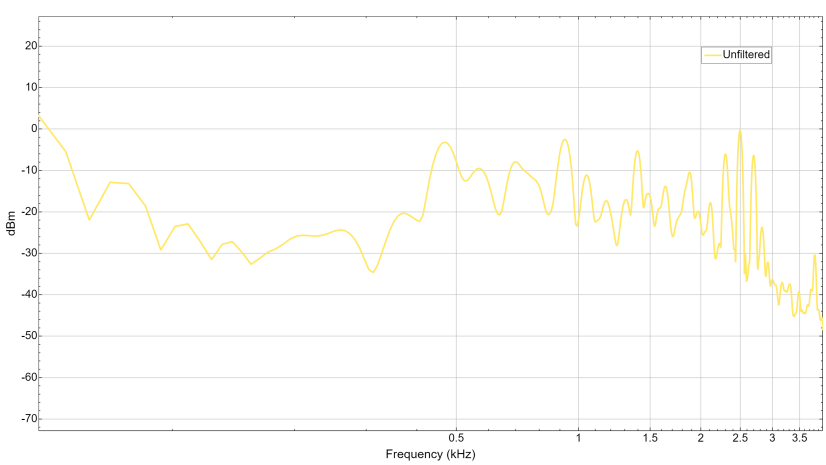


Figure 1: Unfiltered Signal

Based on this analysis,we determined:

1. W_p: frequency where signal is still strong and we want less than 1 dB attenuation (i.e. the useful music is below $1500\,Hz \quad \longrightarrow \quad W_p = 1500\,Hz$).

2. W_s: frequency beyond which signal is mostly noise and needs more than 50 dB attenuation (i.e. the noise signal starts at $2000\,Hz \quad \longrightarrow \quad W_s = 2000\,Hz$).

2 Filtering with a Butterworth IIR filter

```
[LP Butterwhorth Filter]
Minimum Butterworth Filter Order: 16
3dB Cut-off Frequency (normalized): 0.3879
Numerator coefficients (b):
  Columns 1 through 13

    0.0000    0.0001    0.0004    0.0020    0.0065    0.0156    0.0286    0.0409    0.0460    0.0409    0.0286    0.0156    0.0065

  Columns 14 through 17

    0.0020    0.0004    0.0001    0.0000

Denominator coefficients (a):
  Columns 1 through 13

    1.0000   -3.5828    7.9670  -12.2244   14.4074  -13.4054   10.1205   -6.2488    3.1743   -1.3228    0.4492   -0.1225    0.0263



 Columns 14 through 17

  -0.0043    0.0005   -0.0000    0.0000
```

Figure 2: Butterworth Filter Data

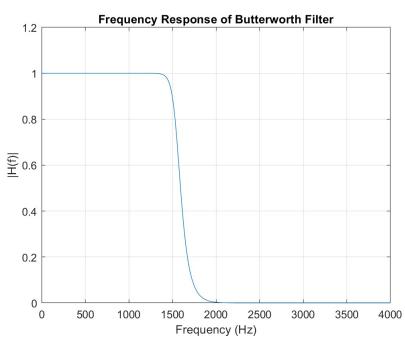


Figure 3

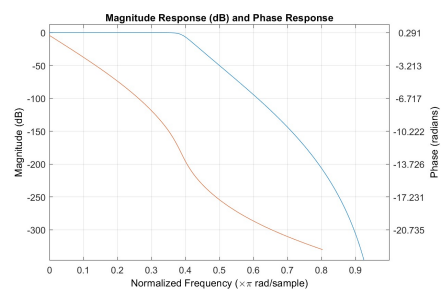


Figure 4

After computing the minimum filter order, 3 dB cut-off frequency, and filter coefficients in MATLAB (Fig.2), and plotting the frequency response (Fig.3) along with the magnitude and phase responses (Fig. 4), we updated the Simulink model with a Discrete Filter block loaded with the calculated filter coefficients. This allowed us to observe the resulting filtered signal.

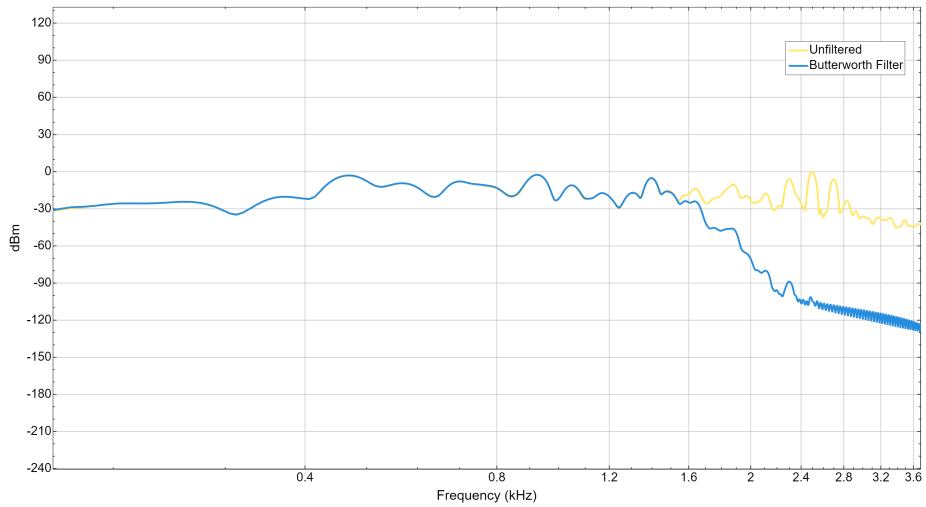


Figure 5: Filtered Signal with Butterworth IIR Filter

3 Filtering with a Chebyshev IIR filter

We repeated the same procedure for this type of filter.

```
[Chebyshev Filter]
Order: 8
3dB cutoff (normalized): 0.3750
b_cheb =
    0.0002    0.0013    0.0046    0.0092    0.0115    0.0092    0.0046    0.0013    0.0002

a_cheb =
    1.0000   -4.4600   10.3665  -15.6144   16.4231  -12.2618    6.3403   -2.0859    0.3393
```

Figure 6: Chebyshev Filter Data

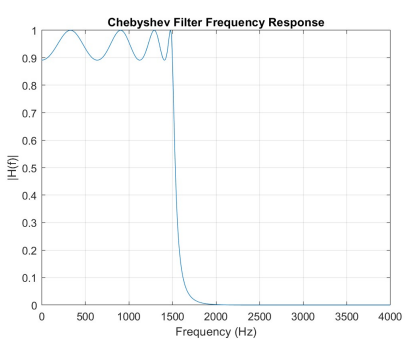


Figure 7

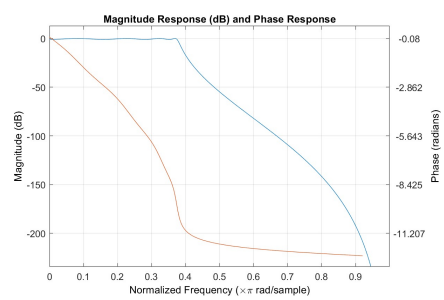


Figure 8

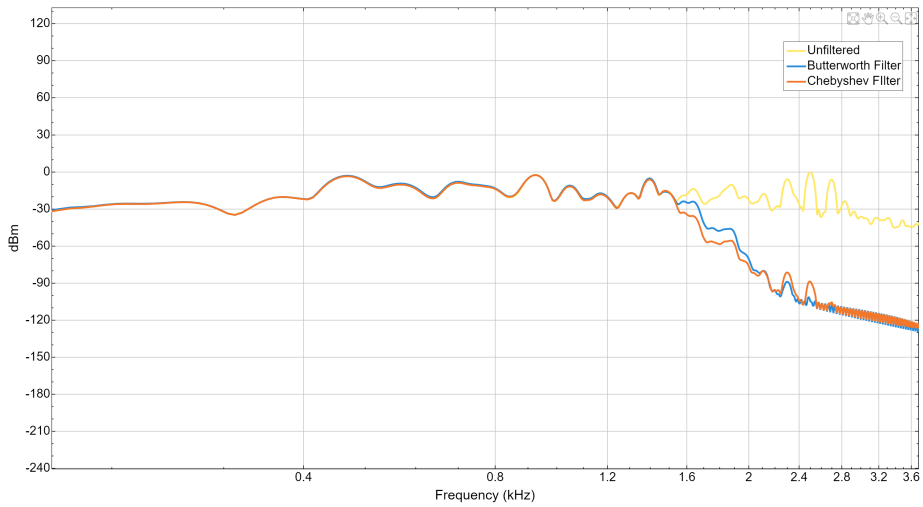


Figure 9: Filtered Signal with Chebyshev IIR Filter

4 Filtering with a FIR filter - The windowing method

Now we want to design a FIR low-pass filter whose frequency response closely matches the previously designed Butterworth and Chebyshev IIR filters.

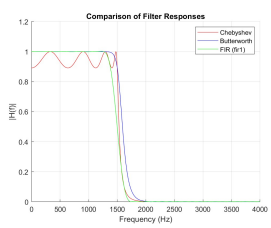


Figure 10: N=60

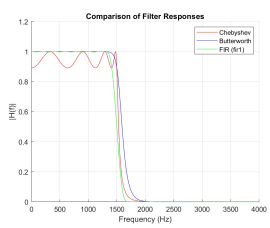


Figure 11: N=80

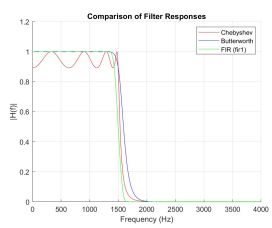


Figure 12: N=100

As observed, increasing the filter order enhances its accuracy, bringing the response closer to that of an ideal IIR filter. However, this improvement in accuracy comes at the cost of increased computational complexity. Therefore, we selected $N = 60$ as a suitable compromise between performance and efficiency.

```
FIR coefficients (b_fir):
  Columns 1 through 13

   -0.0006    0.0003    0.0010    0.0005   -0.0010   -0.0017    0.0000    0.0025    0.0024   -0.0016   -0.0049   -0.0023    0.0050

  Columns 14 through 26

    0.0077   -0.0000   -0.0106   -0.0095    0.0060    0.0181    0.0080   -0.0173   -0.0265    0.0000    0.0370    0.0342   -0.0229

  Columns 27 through 39

   -0.0764   -0.0397    0.1114    0.2933    0.3749    0.2933    0.1114   -0.0397   -0.0764   -0.0229    0.0342    0.0370    0.0000

  Columns 40 through 52

   -0.0265   -0.0173    0.0080    0.0181    0.0060   -0.0095   -0.0106   -0.0000    0.0077    0.0050   -0.0023   -0.0049   -0.0016

  Columns 53 through 61

    0.0024    0.0025    0.0000   -0.0017   -0.0010    0.0005    0.0010    0.0003   -0.0006
```

Figure 13: FIR 1 Filter Data (with N=60)

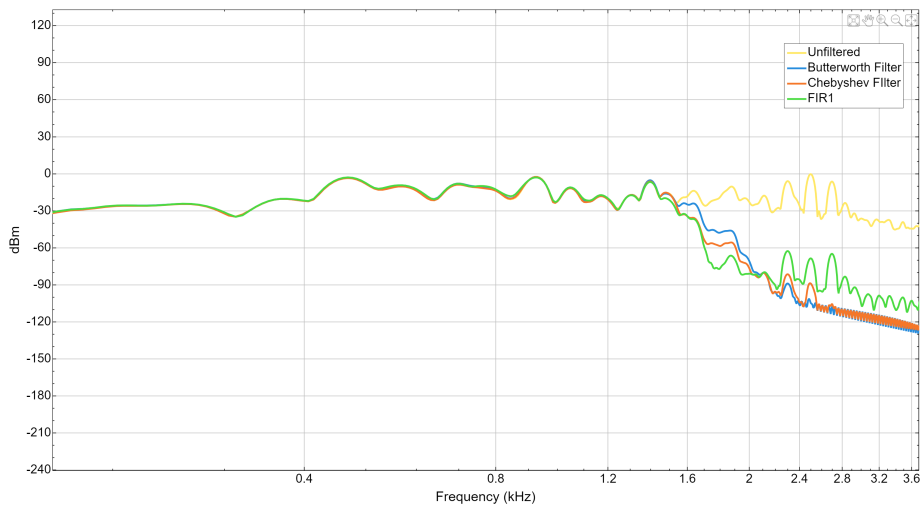


Figure 14: Filter Signal with FIR Filter using windowing method

5 Filtering with an FIR filter - The frequency sampling method

In this case as well, we chose $N = 60$ as it offers a good balance between accuracy and computational efficiency. As shown, beyond $N = 60$, the improvement in accuracy becomes marginal and does not justify the additional computational cost.

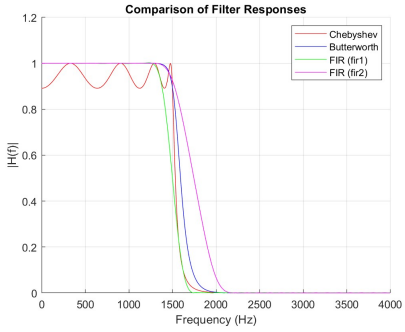


Figure 15: N=60

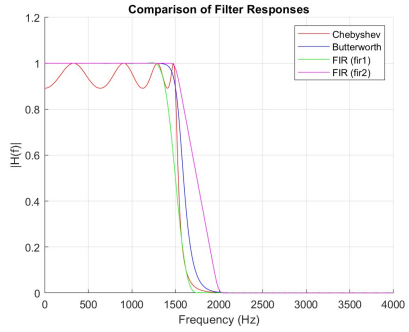


Figure 16: N=200

```
FIR2 Coefficients:
  Columns 1 through 13

    0.0000   -0.0001   -0.0001    0.0001    0.0002   -0.0001   -0.0005   -0.0001    0.0007    0.0004   -0.0006   -0.0007    0.0003

  Columns 14 through 26

    0.0004    0.0000    0.0009    0.0007   -0.0031   -0.0039    0.0049    0.0108   -0.0036   -0.0218   -0.0050    0.0356    0.0276

  Columns 27 through 39

   -0.0494   -0.0810    0.0597    0.3093    0.4365    0.3093    0.0597   -0.0810   -0.0494    0.0276    0.0356   -0.0050   -0.0218

  Columns 40 through 52

   -0.0036    0.0108    0.0049   -0.0039   -0.0031    0.0007    0.0009    0.0000    0.0004    0.0003   -0.0007   -0.0006    0.0004

  Columns 53 through 61

    0.0007   -0.0001   -0.0005   -0.0001    0.0002    0.0001   -0.0001   -0.0001    0.0000
```

Figure 17: FIR 2 Filter Data (with N=60)

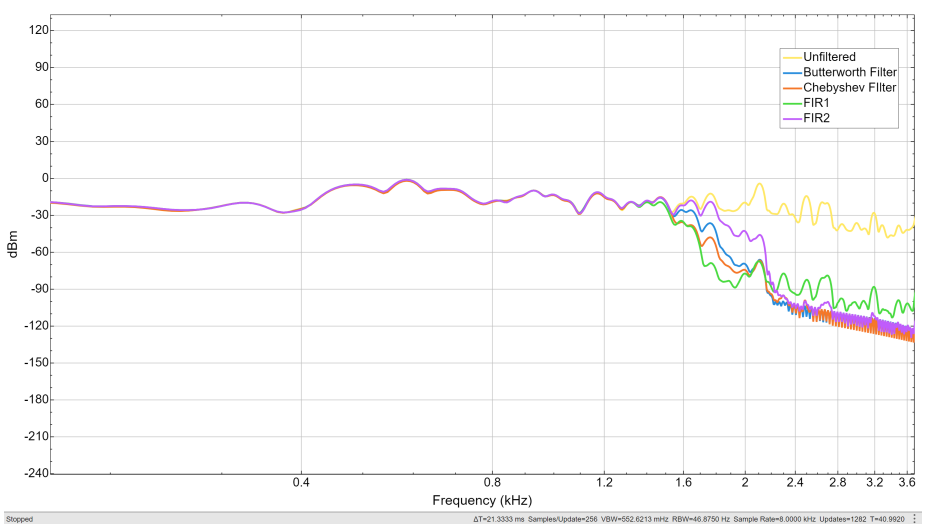


Figure 18: Filtering comparison among all filters

6 On the use of Filter Design & Analysis Tool

MATLAB's `filterDesigner` tool provides an interactive GUI to design and analyze filters. Let's walk through the full process step by step to design an effective digital filter to clean the noisy music signal. To design a BUtterworth filter we set up the following filter specs:

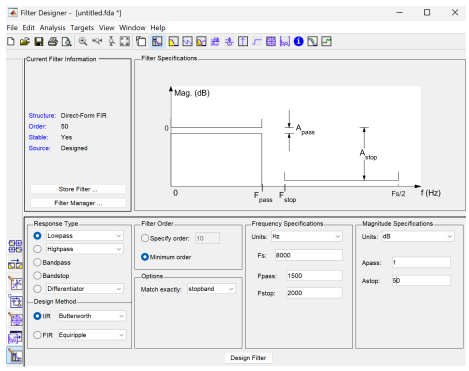
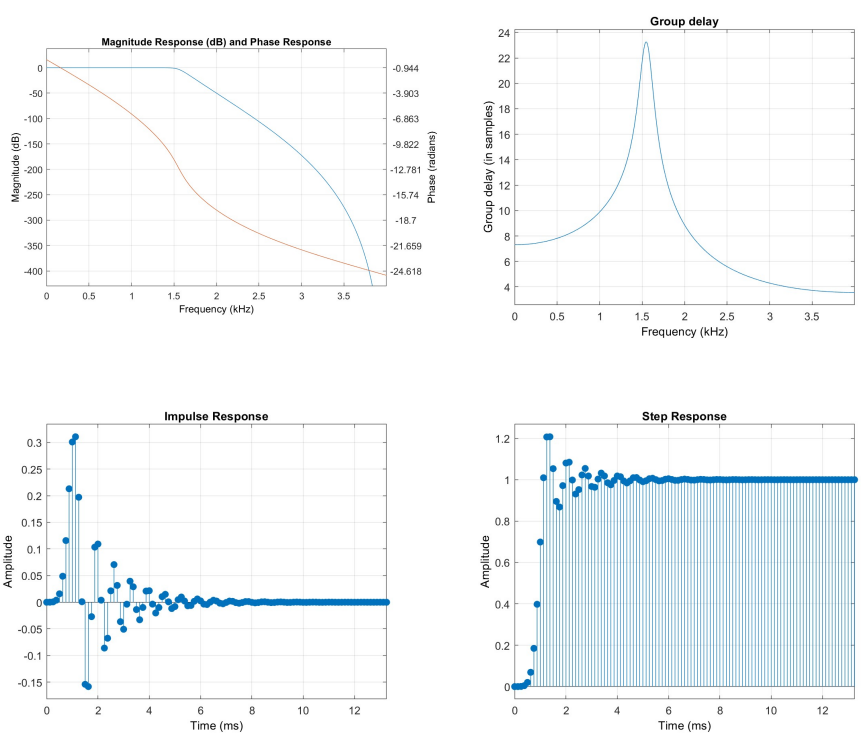
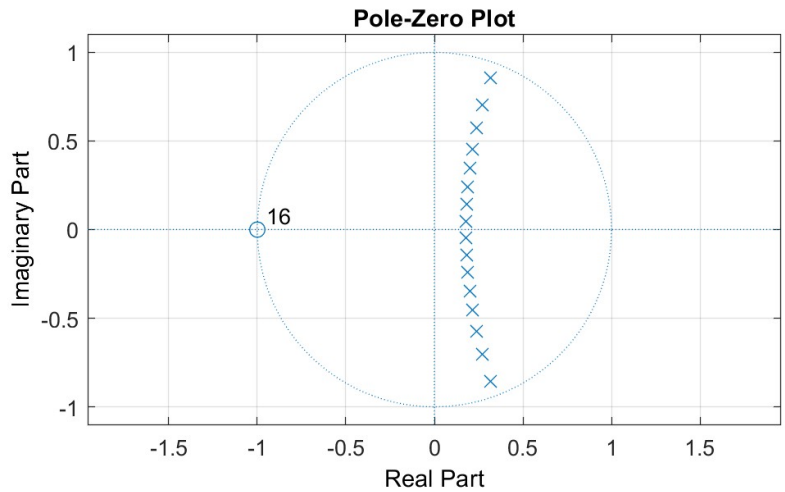


Figure 19: Filter specifications on the GUI of `filterDesigner` command

After clicking on "Design Filter" we're able to see all the plots:





To recover the coefficients of the filter we went to: `Analysis` $\longrightarrow$ `Filter Coefficients` and then to `File` $\longrightarrow$ `Export to Simulink Model`. From Simulink, we uploaded the music file again, and then using the spectrum analyzer block, we compared the result with the previous obtained result as we can see in Fig.20. The results completely match!

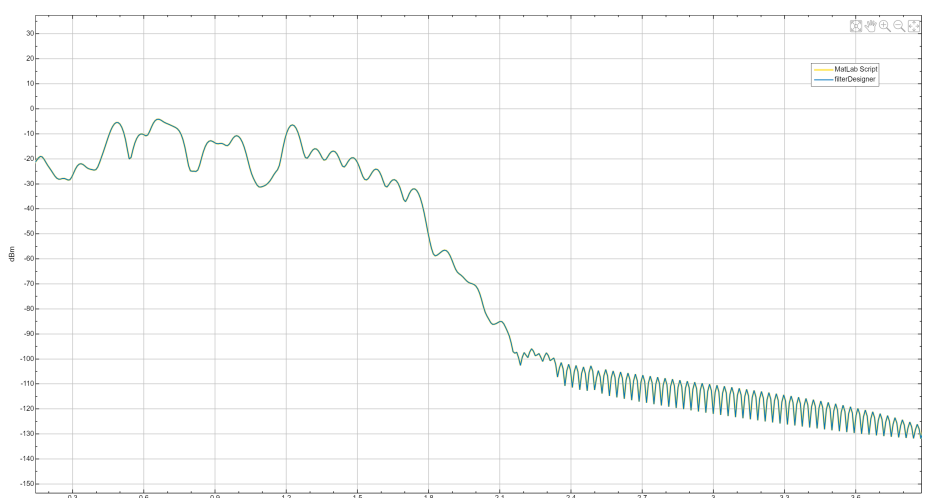


Figure 20: Filtered Signal using filterDesigner

Appendix

```
%==============================
% S I G N A L   S P E C T R U M
%==============================
% Load the signal
[x, Fs] = audioread('music_bruitee.wav');  % Fs = 8000 Hz
x = x(:,1);  % Use one channel if stereo

% Perform FFT
n = length(x);
X = abs(fft(x));
f = (0:n-1)*(Fs/n);  % Frequency axis

% Plot the frequency spectrum
figure;
plot(f, X);
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Frequency Spectrum of Noisy Music');
xlim([0 Fs/2]);  % Show only up to Nyquist
grid on;

%=========================================================
% L P   B U T T E R W O R T H   F I L T E R   D E S I G N
%=========================================================

% Define filter specs
Wp = 1500 / (Fs/2);  % Normalized passband edge (0.375)
Ws = 2000 / (Fs/2);  % Normalized stopband edge (0.5)
Rp = 1;              % Passband ripple in dB
Rs = 50;             % Stopband attenuation in dB

% Estimate minimum order and 3dB cut-off
[N, Wn] = buttord(Wp, Ws, Rp, Rs);

% Display results
fprintf('[LP Butterwhorth Filter]\n');
fprintf('Minimum Butterworth Filter Order: %d\n', N);
fprintf('3dB Cut-off Frequency (normalized): %.4f\n', Wn);

% Design the Butterworth filter
[b, a] = butter(N, Wn);

% Display the filter coefficients
disp('Numerator coefficients (b):');
disp(b);
disp('Denominator coefficients (a):');
disp(a);

% Compute the frequency response
[Rf, fr] = freqz(b, a, 1024, Fs);  % 1024 points across 0-Fs/2 Hz

% Plot the magnitude of the frequency response
figure;
plot(fr, abs(Rf));
xlabel('Frequency (Hz)');
ylabel('|H(f)|');
title('Frequency Response of Butterworth Filter');
```

```
% Plot frequency response
fvtool(b, a);

% Optional: apply filter and listen
y = filter(b, a, x);
sound(y, Fs);  % Play filtered sound


%=========================================================
% C H E B Y S H E V   F I L T E R   D E S I G N
%=========================================================

[N_cheb, Wn_cheb] = cheb1ord(Wp, Ws, Rp, Rs);
fprintf('[Chebyshev Filter]\n');
fprintf('Order: %d\n', N_cheb);
fprintf('3dB cutoff (normalized): %.4f\n', Wn_cheb);

[b_cheb, a_cheb] = cheby1(N_cheb, Rp, Wn_cheb);
disp('b_cheb ='); disp(b_cheb);
disp('a_cheb ='); disp(a_cheb);

% Plot frequency response
[Hc, fc] = freqz(b_cheb, a_cheb, 1024, Fs);
figure;
plot(fc, abs(Hc));
xlabel('Frequency (Hz)');
ylabel('|H(f)|');
title('Chebyshev Filter Frequency Response');
grid on;

% Plot frequency response
fvtool(b_cheb, a_cheb);

% Apply filter
y_cheb = filter(b_cheb, a_cheb, x);
sound(y_cheb, Fs);

%=========================================================
% F I R   F I L T E R   D E S I G N  (windowing method)
%=========================================================

Wn_fir = 1500 / (Fs / 2);
N_fir = 60;

b_fir = fir1(N_fir, Wn_fir);
disp('FIR coefficients (b_fir):'); disp(b_fir);

[H_fir, f_fir] = freqz(b_fir, 1, 1024, Fs);

% Plot comparison
figure;
hold on;
plot(fc, abs(Hc), 'r', 'DisplayName', 'Chebyshev');
plot(fr, abs(Rf), 'b', 'DisplayName', 'Butterworth');
plot(f_fir, abs(H_fir), 'g', 'DisplayName', 'FIR (fir1)');
xlabel('Frequency (Hz)');
ylabel('|H(f)|');
title('Comparison of Filter Responses');
```

```
% Filter audio
y_fir = filter(b_fir, 1, x);
sound(y_fir, Fs);  % Play filtered FIR result


%==============================================================
% F I R   F I L T E R   D E S I G N  (frequency sampling method)
%==============================================================

N_fir2 = 60;  % Try 40, 60, 80 etc.
F = [0 0.375 0.5 1];  % Normalized frequency vector
A = [1 1 0 0];        % Desired amplitude

b_fir2 = fir2(N_fir2, F, A);
disp('FIR2 Coefficients:'); disp(b_fir2);

[H_fir2, f_fir2] = freqz(b_fir2, 1, 1024, Fs);

% Plot comparison
figure;
hold on;
plot(fc, abs(Hc), 'r', 'DisplayName', 'Chebyshev');
plot(fr, abs(Rf), 'b', 'DisplayName', 'Butterworth');
plot(f_fir, abs(H_fir), 'g', 'DisplayName', 'FIR (fir1)');
plot(f_fir2, abs(H_fir2), 'm', 'DisplayName', 'FIR (fir2)');
xlabel('Frequency (Hz)');
ylabel('|H(f)|');
title('Comparison of Filter Responses');
legend;
grid on;

% Apply and listen
y_fir2 = filter(b_fir2, 1, x);
sound(y_fir2, Fs);
```

Figure 21: MatLab Script

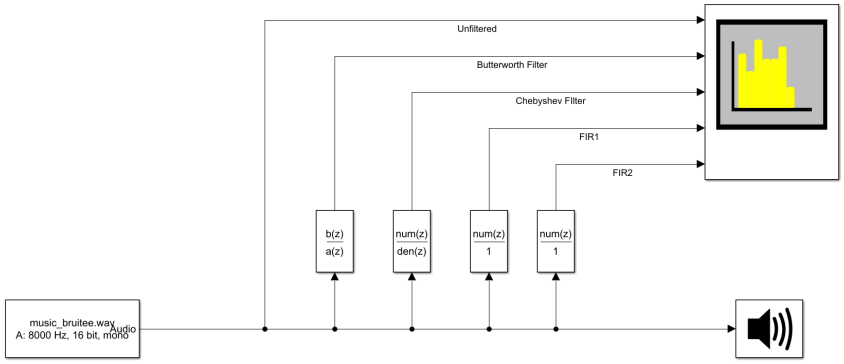


Figure 22: Simulink Scheme

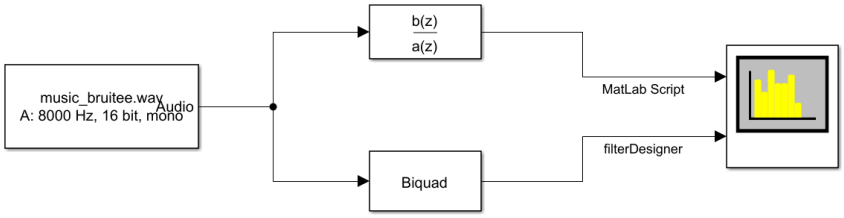


Figure 23: Simulink Scheme with `filterDesigner`